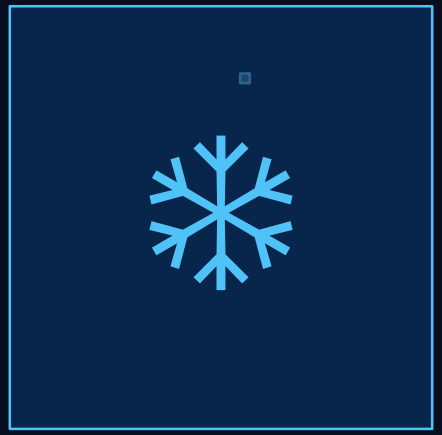

FROZEN FRONTIER

An Alien Ice World Explorer

Student: Fouad Bellili

Repository: github.com/FouadBellili/ICG_PROJECT

Live Demo: fouadbellili.github.io/ICG_PROJECT/

Course: Introduction to Computer Graphics 2025/2026

THE PROJECT

What is it?

- First-person 3D exploration game on an alien frozen planet
- Explore infinite procedurally generated icy terrain
- Discover 5 mysterious glowing artefacts with proximity lore
- Survive wind, snowfall and rocky obstacles
- Built with Three.js r168 + PointerLockControls addon
- No external game engine — pure WebGL via Three.js
- Three.js version: r168 (imported via CDN importmap)

MODELS & SCENE GRAPH

Models

Ice terrain mesh	PlaneGeometry, height-displaced per chunk
Ice overlay	Semi-transparent plane on top of terrain
Rocks	IcosahedronGeometry + per-vertex noise deformation
Ice spires	ConeGeometry (5–8 sides), random tilt
Artefacts	OctahedronGeometry, emissive material
Snow particles	3 × BufferGeometry Points layers
Planet disc	2 × CircleGeometry (background)
Star field	BufferGeometry, 6000 random Points

Scene Graph

ILLUMINATION · MATERIALS · TEXTURES

Light Sources

AmbientLight
Deep space blue, intensity 1.0

DirectionalLight (sun)
Cold blue-white, shadows 2048px, tracks player

Fill DirectionalLight
Blue rim, opp. direction

PointLights ×5
One per artefact, pulsing, range 18

Materials

Ice terrain
MeshStandardMaterial, rough 0.48, metal 0.22

Ice overlay
Transparent, AdditiveBlending

Rocks
MeshStandard + roughness + normal maps

Spires
MeshStandard, transparent 0.82

Artefacts
Emissive MeshStandard, pulsing intensity

Textures

Ice roughness map
Canvas API, multi-octave noise, 512px

Ice normal map
Finite differences height-field, 512px

Rock roughness map
High-freq noise, 512px

Rock normal map
Sharp surface details, 512px

Snowflake sprite
Radial gradient, 64px, transparent edge

No custom shaders — all textures generated procedurally at runtime via Canvas API (zero external texture files)

ANIMATION

Animations

- **Snow particle system (3 layers)**
  - · Per-particle velocity (0.012–0.042 units/frame × spdMult)
  - · Wind vector: sin/cos sum with slow time drift
  - · Per-flake flutter: sin(t·0.35 + phase_i)
  - · Wrap-around: reset to top when y < –2
- **Artefact pulse**
  - · emissiveIntensity = 0.4 + 0.7·(0.5 + 0.5·sin(t·1.8 + idx·1.3))
  - · Y-rotation: θ = t·0.5 + idx
  - · PointLight intensity synced: 1.0 + 1.5·pulse
- **Sun tracking**
  - · DirectionalLight follows camera XZ position
  - · Subtle intensity oscillation: 2.5 + 0.15·sin(t·0.3)
- **Star field**
  - · Slow Y-axis rotation: stars.rotation.y = t·0.005

USER INTERACTION

Camera

Type: PerspectiveCamera (FoV 75°, near 0.1, far 500)
Why: First-person immersion; PointerLockControls attached to renderer.domElement

Keyboard Controls

WASD Move forward / left / back / right

Space Jump (impulse applied to velocityY)

ESC Release pointer lock / pause

Click Lock cursor and enter exploration mode

Mouse

- PointerLock API — cursor hidden during play
- mousemove → camera yaw + pitch
- Sensitivity: pointerSpeed = 0.65
- Unlock on ESC → pause screen shown

HUD / UI

- Overlay screen: click-to-enter + control hints
- In-game HUD: title + control reminder
- Crosshair: centered "+" during play
- Lore panel: fades in when near artefact (<12 units)
- Frost vignette: pure CSS overlay (no interaction)

DEVELOPMENT

Environment & Organization

Editor	VS Code
Version control	Git + GitHub
Runtime	Single HTML file, no build step
Modules	Three.js via CDN importmap (ES modules)
Deployment	GitHub Pages
Browser tested	Chrome, Firefox

Key Implementation Details

- Chunk system: lazy-load/unload 7×7 grid around player
- Seeded RNG (Mulberry32) ensures deterministic terrain per chunk
- Collision: AABB-circle pushback for rocks + terrain floor clamp

Difficulties

- Seeded RNG needed to avoid rock/spire re-randomization on chunk reload
- Snow flutter: keeping particles inside spread range while simulating wind drift
- Audio: Web Audio API requires user gesture before context resume
- Texture generation: normal map finite-differences need tuning per-material

USE OF AI

Tools used

Anthropic Claude (claude.ai) — Primary AI assistant used throughout the project

Tasks & degree of use

HIGH	**Procedural terrain noise function**	Claude suggested the 4-octave smooth-noise height formula and UV scaling factors
HIGH	**Seeded PRNG (Mulberry32)**	Claude provided the Mulberry32 PRNG implementation and chunk-seed hash
HIGH	**Web Audio sound design**	Claude designed the wind ambience (bandpass+LFO), breathing rhythm and footstep envelope
HIGH	**Canvas-API texture generation**	Claude wrote the finite-difference normal map and multi-octave roughness map generators
MED	**Physics & collision**	Claude suggested the cylinder-radius pushback collision logic and gravity/jump integration
MED	**Artefact pulse animation**	Claude wrote the sin-based emissive intensity formula
MED	**Code debugging & structure**	Claude helped debug chunk unload memory leaks (geometry.dispose) and reviewed code

PERFORMANCE & CONCLUSIONS

Performance

Chunk streaming	Only 7×7 grid loaded; chunks beyond VIEW_RANGE=3 removed and geometries disposed
Geometry reuse	Shared materials (terrainMat, rockMat, spireMat) across all chunks
Pixel ratio cap	renderer.setPixelRatio(min(devicePixelRatio, 2))
Shadow map	Single DirectionalLight shadow, 2048px, local frustum
Snow optimisation	3 separate particle layers (3500 / 1800 / 450)
Browser compat.	Chrome ✓ Firefox ✓ Safari (pointer lock limited) Mobile: limited

Key Features

- Fully procedural world — infinite, deterministic, no assets
- Dynamic layered snowfall with wind simulation
- Spatial audio system via Web Audio API (no library)
- Proximity-based lore discovery system
- Real-time procedural texture generation (Canvas API)

What I would do differently

- Add touch / gamepad controls for mobile compatibility
- Implement LOD (Level of Detail) for distant terrain chunks
- Use a proper instanced mesh for rocks/spires (better GPU performance)
- Add day/night cycle with dynamic sky gradient

REFERENCES

Documentation

Three.js Documentation (r168). (2024). Three.js – JavaScript 3D Library. https://threejs.org/docs/

MDN Web Docs. (2024). Web Audio API. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API

MDN Web Docs. (2024). Pointer Lock API. https://developer.mozilla.org/en-US/docs/Web/API/Pointer_Lock_API

Techniques & Tutorials

Bourke, P. (1999). Interpolation Methods — smooth-step noise algorithm. http://paulbourke.net/miscellaneous/interpolation/

Squirrel Eiserloh. (2017). Fast & Furious PRNG — Mulberry32 hash. GDC Math for Game Programmers talk.

Scratchapixel. (2022). Procedural Textures — noise functions. https://www.scratchapixel.com/

AI Tools

Anthropic. (2026). Claude [Large language model]. https://claude.ai

Course Materials

Madeira, J. (2025/2026). Introduction to Computer Graphics — lecture slides & examples. Universidade de Aveiro.